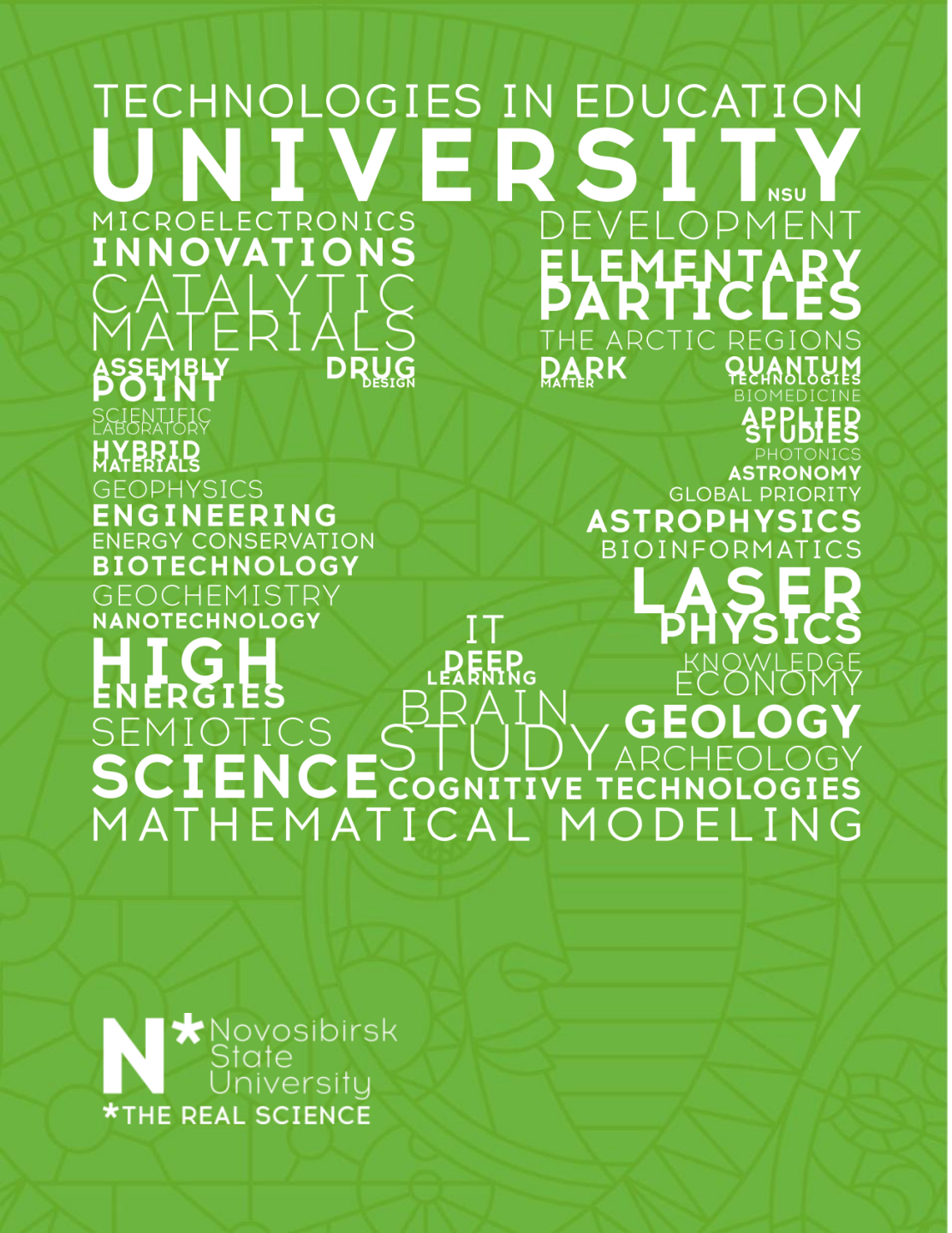


Базовые методы ИИ
Семинар 10

Глушенко Андрей Валерьевич
ФФ НГУ

Практические аспекты

Метод опорных векторов

Разделение на обучающую, валидационную и тестовую выборки

Кросс-валидация

Метрики

Метод опорных векторов (SVM)

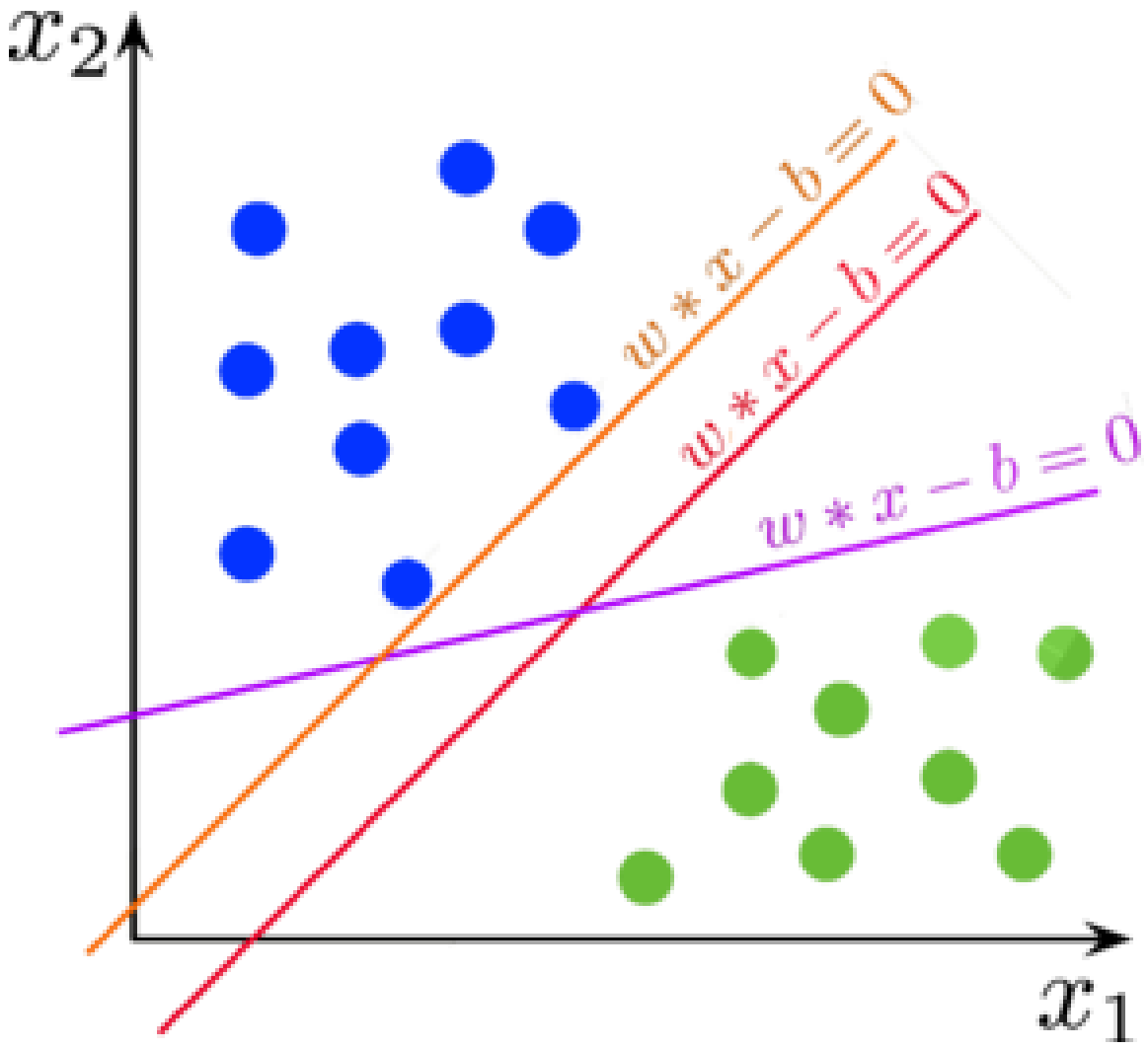
Метод опорных векторов (Support Vector Machine или **SVM**) — это один из самых популярных и эффективных алгоритмов обучения «с учителем», который используется для задач **классификации** и регрессии.

SVM. Основная идея

Представьте, что на плоскости перемешаны точки двух цветов. Задача SVM — провести между ними линию (или гиперплоскость в многомерном пространстве) так, чтобы разделить их «наилучшим» образом.

«Наилучшим» в данном случае считается такое разделение, при котором зазор (margin) между классами будет максимальным. Чем шире эта «пустая дорога» между группами данных, тем увереннее алгоритм будет классифицировать новые объекты.

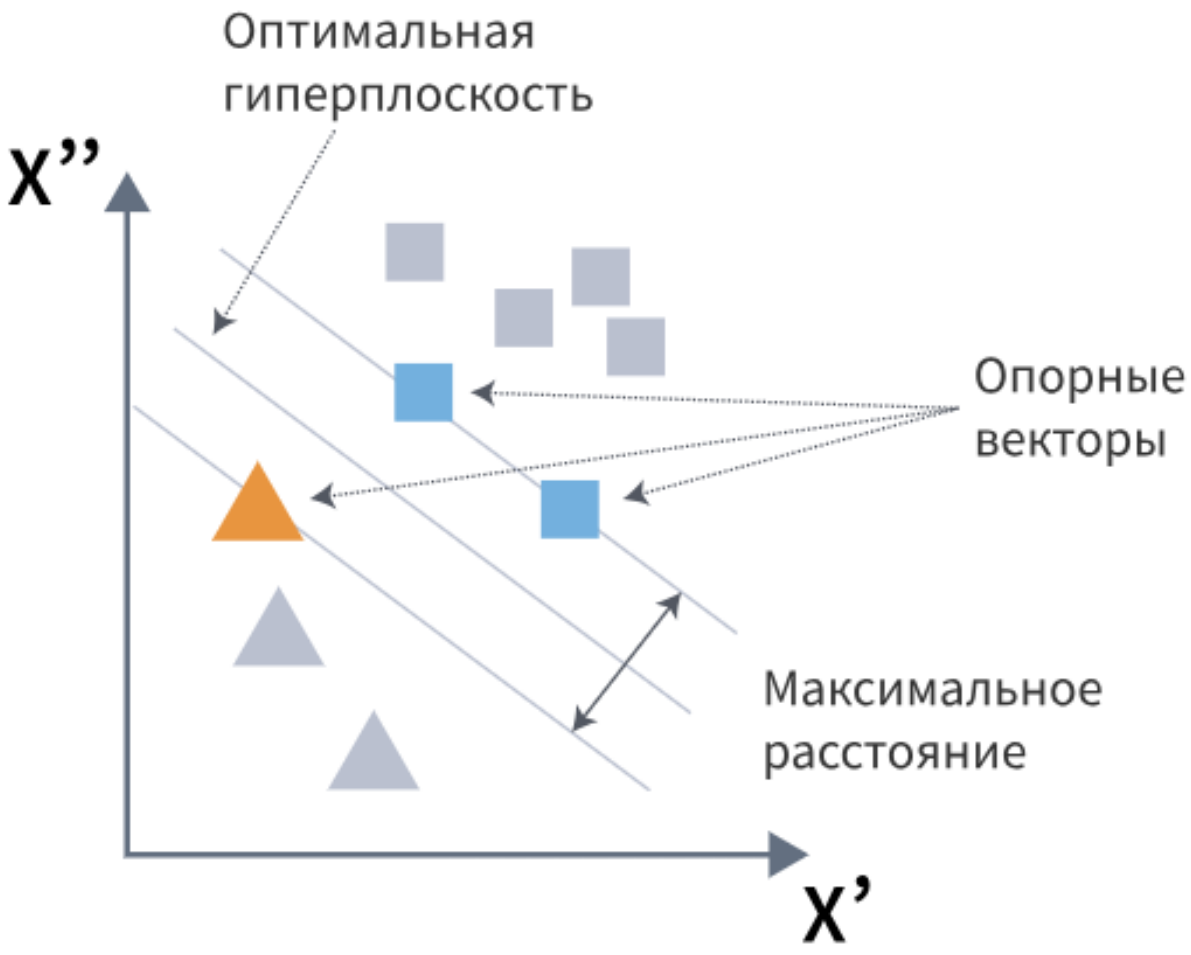
SVM. Основная идея



Основные понятия

- **Опорные векторы:** Это «крайние» точки в наборах данных, которые лежат ближе всего к разделяющей линии. Именно они определяют положение границы. Если их убрать, граница сдвинется, поэтому они и называются опорными.
- **Гиперплоскость:** Это сама граница раздела. В 2D — это линия, в 3D — плоскость, а в более высоких размерностях — гиперплоскость.
- **Ядерный трюк (Kernel Trick):** Если данные невозможно разделить прямой линией (например, один класс внутри другого), SVM переносит их в пространство более высокой размерности, где разделение становится возможным.

Основные понятия



Преимущества

- **Точность:** SVM отлично работает на выборках с четкой границей разделения.
- **Эффективность:** Он справляется даже тогда, когда количество признаков (колонок в таблице) больше, чем количество самих объектов.
- **Устойчивость:** Благодаря фокусу на зазоре, метод менее склонен к ошибкам из-за случайных выбросов, которые находятся далеко от границы.

Алгоритм SVM

1. Поиск разделяющей гиперплоскости

Сначала алгоритм ищет все возможные варианты границ (линий или плоскостей), которые могут разделить ваши данные на два класса.

В n-мерном пространстве разделяющая граница описывается линейным уравнением:

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{x} + \boldsymbol{b} = \boldsymbol{0}, \text{где:}$$

x — вектор признаков объекта.

w — вектор весов (нормаль к гиперплоскости), определяющий её ориентацию.

b — смещение (bias), определяющее расстояние от начала координат.

Алгоритм SVM

Условие классификации

Для каждого объекта x_i из обучающей выборки мы хотим, чтобы алгоритм выдавал правильную метку y_i(принимает значения +1 или -1):

$$w^T x_i + b \ \geq 1, \text{если} y_i = +1$$
$$w^T x_i + b \ \leq 1, \text{если} y_i = -1$$

Это можно объединить в одно неравенство:

$$y_i(w^T x_i + b) \ \geq 1$$

Алгоритм SVM

2. Максимизация зазора (Geometric Margin)

Из всех найденных границ SVM выбирает ту, у которой самый широкий зазор.

- **Зазор** — это расстояние от границы до ближайших точек каждого класса.
- Алгоритм стремится сделать эту «дорогу» максимально широкой, чтобы минимизировать риск ошибки при появлении новых данных.

Геометрическое расстояние от точки до гиперплоскости равно $\frac{1}{\|w\|}$. Общий зазор между двумя классами равен $\frac{2}{\|w\|}$.

Чтобы сделать зазор максимальным, нам нужно минимизировать норму вектора весов $\|w\|$.

Математически задача оптимизации выглядит так:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \text{ для всех } i \text{ таких, что } y_i(w^T x_i + b) \geq 1$$

Алгоритм SVM

3. Определение опорных векторов

В процессе выбора границы алгоритм находит те самые точки, которые стоят вплотную к «обочине» зазора. Это и есть опорные векторы.

- Все остальные точки данных, которые находятся дальше, алгоритму не важны — они не влияют на положение границы.

Алгоритм SVM

4. Мягкий зазор (Soft Margin)

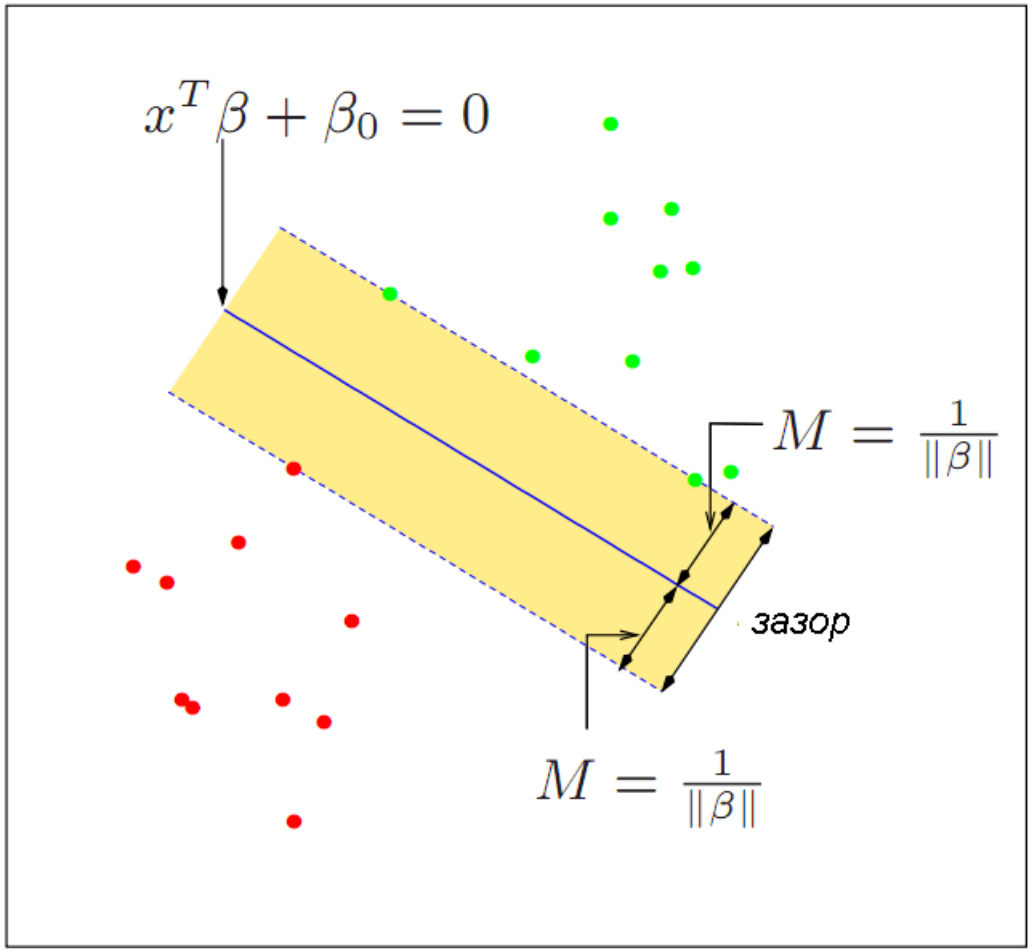
В реальности данные редко разделяются идеально. Чтобы позволить алгоритму допускать небольшие ошибки, вводят переменные ξ_i (slack variables) и параметр регуляризации C :

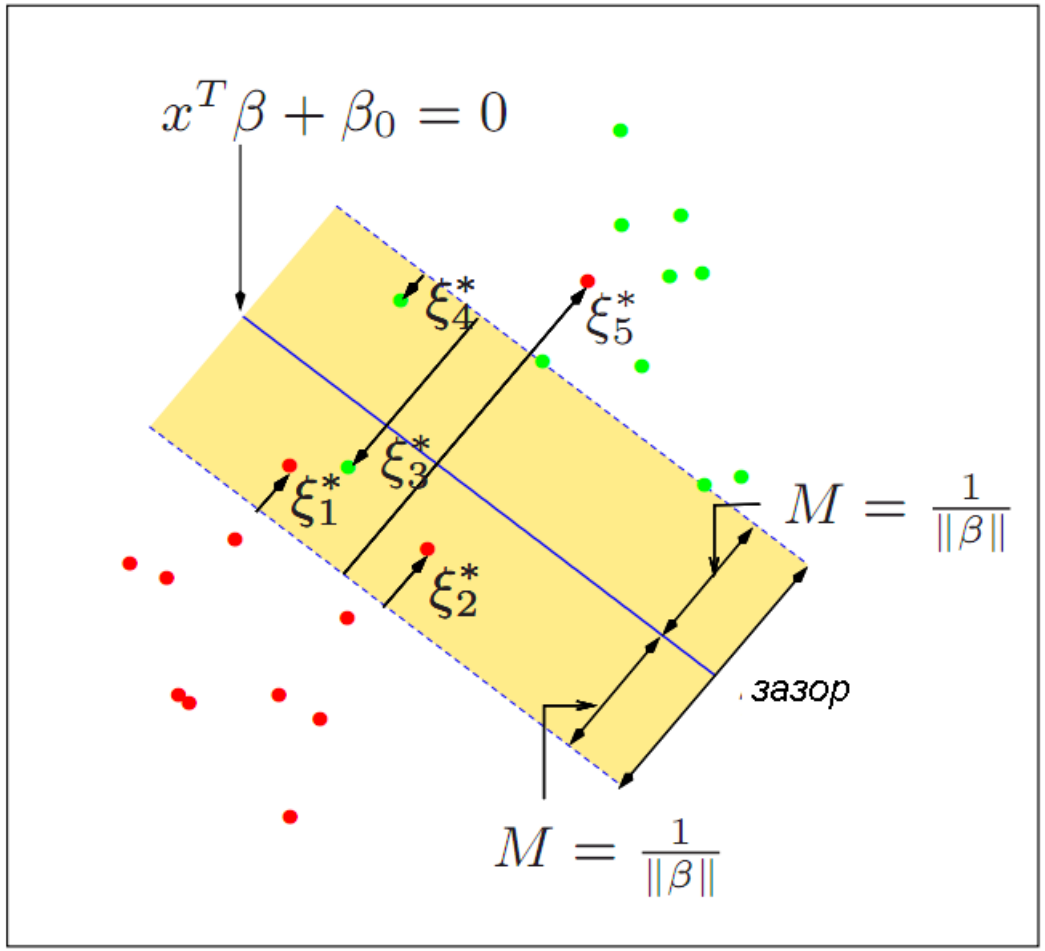
$$\min_{w,b} \frac{1}{2}\|w\|^2 + \mathrm{C}\sum_{i=1}^{n} \xi_i$$

- C большой: алгоритм штрафует за любую ошибку (жесткая граница).
- C маленький: алгоритм допускает ошибки ради более широкого зазора (мягкая граница).

Алгоритм SVM

4. Мягкий зазор (Soft Margin)





Алгоритм SVM

5. Переход в высшее пространство (Kernel Trick)

Если данные «линейно неразделимы» (например, один круг внутри другого), алгоритм применяет математическую функцию — ядро. Оно переносит данные в новое измерение (например, из 2D в 3D), где их можно легко разделить одной ровной плоскостью.

Функции ядер

Когда данные нельзя разделить прямой линией или плоскостью в их текущем виде, на помощь приходят **функции ядер (Kernels)**.

Их магия в том, что они вычисляют «близость» точек в пространстве более высокой размерности, не выполняя само преобразование координат (это экономит кучу вычислительных ресурсов).

Линейное ядро (Linear Kernel)

Это самый простой вариант. Оно не меняет размерность и просто ищет прямую линию.

Формула:

$$K(x, x_i) = \sum_{i=1}^{n} (x \cdot x_i)$$

Когда использовать: Если у вас очень много признаков (например, классификация текстов), данные часто уже разделимы линейно.

Полиномиальное ядро (Polynomial Kernel)

Оно представляет данные в виде комбинаций исходных признаков (возводит их в степень). Представьте, что вы искривляете пространство, чтобы «обернуть» границу вокруг группы точек.

Формула:

$$K(x, x_i) = \sum_{i=1}^{n} (x \cdot x_i + r)^d$$

Параметры: $\boldsymbol{d}$ — степень полинома. Чем выше d, тем сложнее (извилистее) граница.

Когда использовать: В задачах распознавания образов.

RBF (Radial Basis Function) / Гауссово ядро

Самое популярное и мощное ядро «по умолчанию». Оно работает так, будто вокруг каждой точки создается «облако влияния». Оно теоретически переносит данные в пространство бесконечной размерности.

Формула:

$$K(x, x_i) = e^{-\gamma \|x - x_i\|^2}$$

Параметры:

- **Большая γ** — модель учитывает только ближайшие точки (граница будет очень «дерганой»).
- **Маленькая γ** — модель смотрит на данные более глобально (граница будет плавной).

Когда использовать: Когда вы не знаете, какую форму имеет граница между классами.

Сигмоидное ядро (Sigmoid Kernel)

По принципу работы оно похоже на функцию активации в нейронных сетях.

Формула:

$$K(x, x_i) = \tanh(\alpha x^T x_i + c)$$

Когда использовать: Пришло из теории нейросетей, но на практике в SVM используется реже, чем RBF.

Сравнение, выбор ядра

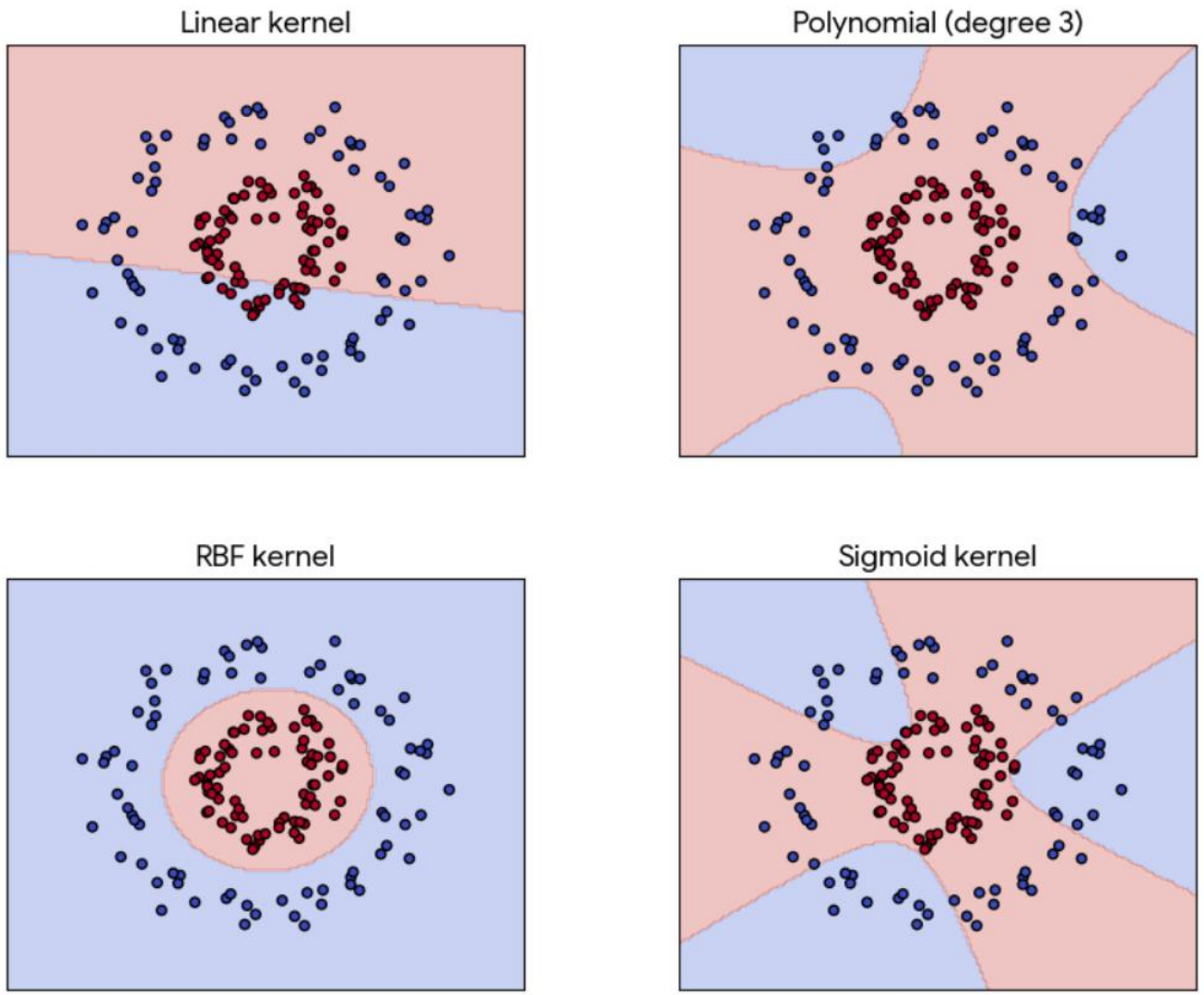
- ✓ Начните с линейного, если признаков больше, чем объектов.
- ✓ Попробуйте RBF, если признаков мало, а объектов много.
- ✓ Полиномиальное стоит пробовать, если есть подозрение на конкретные зависимости признаков.

Чтобы наглядно увидеть разницу, представим сложный случай: **один класс данных находится внутри другого**. Обычная прямая линия здесь бессильна.

Сравнение, выбор ядра

- **Linear (Линейное):** Пытается провести прямую черту. Как видно, оно полностью проваливает задачу, так как данные имеют круговую структуру. Синие точки внутри красного кольца остаются неразбитыми.
- **Polynomial:** За счет возведения признаков в степень граница начинает изгибаться. Однако при степени $d = 3$ оно все еще может не идеально описывать круг, создавая более сложные, но не всегда симметричные формы.
- **RBF (Гауссово):** Идеально справляется с задачей. Оно создает замкнутую область вокруг центрального кластера. Это ядро наиболее адаптивно и «обволакивает» данные любой формы.
- **Sigmoid (Сигмоидное):** Пытается имитировать поведение нейронных сетей. В данной задаче оно часто ведет себя непредсказуемо и редко дает хорошую границу для круговых данных.

Сравнение, выбор ядра



Подбор параметра Gamma (γ)

Параметр Gamma (γ) в SVM с ядром RBF определяет «радиус влияния» отдельного опорного вектора. Проще говоря, он говорит алгоритму, насколько далеко распространяется «авторитет» одной точки данных.

Маленькая γ:

- **Радиус:** Очень большой. Каждая точка влияет на классификацию точек, находящихся далеко от неё.
- **Граница:** Получается очень плавной, почти прямой.
- **Риск: Недообучение (Underfitting).** Модель может пропустить важные изгибы в данных и стать слишком простой.

Подбор параметра Gamma (γ)

Параметр Gamma (γ) в SVM с ядром RBF определяет «радиус влияния» отдельного опорного вектора. Проще говоря, он говорит алгоритму, насколько далеко распространяется «авторитет» одной точки данных.

Большая γ:

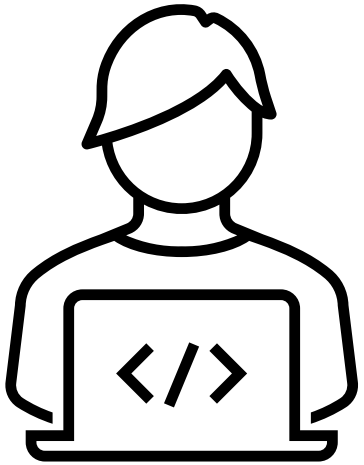
- **Радиус:** Маленький. Точка влияет только на ближайших «соседей».
- **Граница:** Становится очень сложной, извилистой, буквально «облегает» каждую точку.
- **Риск: Переобучение (Overfitting).** Модель выучит шум и случайные колебания в данных вместо общей закономерности.

Как подбирать γ на практике?

Подбор γ всегда идет в связке с параметром C (штраф за ошибки). Основной инструмент для этого — **Grid Search (Поиск по сетке) с кросс-валидацией.**

- **Логарифмическая сетка:** γ обычно меняется в геометрической прогрессии. Начните с широкого диапазона: $10^{-4}, 10^{-3}, 10^{-2}, \dots, 10^{2}, 10^{3}$.
- **Визуализация валидации:** Используйте тепловую карту (Heatmap) точности, где по одной оси — C, по другой — γ. Ищите «пятно», где точность на тесте максимальна.
- **Правило «золотой середины»:** Если граница на графике выглядит как рваные лоскуты вокруг каждой точки — γ слишком велика. Если она почти не гнется там, где это явно нужно — γ слишком мала.

Практика



Ресурсы для изучения ML

Лабораторные работы